University of Asia Pacific



Charity & Volunteer Management System

Course Title: Software Engineering Lab (CSE 314)

Submitted By

Team Members

Md. Mahfuzur Rahman	– Project Manager	23201206
Toma Rani Kabiraj	– Reporting Lead	23201214
Wasikul Hasan Fahim	– Team Lead	22201037

Submitted To

Md. Ashraful Alam
Lecturer
Department of Computer Science and Engineering
University of Asia Pacific

23 August 2026

Contents

Executive Summary **1**

1 Introduction **2**

1.1 Goals and Objectives of the Project 2

1.2 Scope of Work 2

1.3 System Overview 3

1.4 Structure of the Document 3

2 Project Management Plan and Feasibility Analysis **5**

2.1 Project Organization 5

2.1.1 Individual Contribution to the Project 6

2.2 Process Model Used 6

2.2.1 Rationale for Choosing the Lifecycle Model 7

2.3 Risk Analysis 7

2.4 Constraints to Project Implementation 8

2.5 Hardware and Software Resource Requirements 8

2.5.1 Software Requirements 8

2.5.2 Hardware Requirements 8

2.6 Project Timeline and Schedule 9

2.7 Estimated Budget 9

2.8 Social, Cultural, and Environmental Impact of the Project 10

2.8.1 Social Impact 10

2.8.2 Cultural Impact 10

2.8.3 Environmental Impact 10

2.9 Feasibility Analysis 10

2.9.1 Technical Feasibility 11

2.9.2 Economic Feasibility 11

2.9.3 Operational Feasibility 11
2.9.4 Legal and Ethical Feasibility 11
2.9.5 Schedule Feasibility 11

3 Requirement Specifications 12

3.1 Software Requirements Specification (SRS) 12
3.1.1 Introduction 12
3.1.2 Overall Description 13
3.1.3 Functional Requirements 14
3.1.4 Non-Functional Requirements 17
3.1.5 Technical Requirements 18
3.1.6 System Constraints 19
3.1.7 Assumptions and Dependencies 19
3.1.8 Proposed Solution 19
3.1.9 Expected System Output 20
3.1.10 Social Impact 20
3.1.11 Risk Analysis 21
3.2 Architecture Diagram 21
3.3 State Diagram 22
3.4 Activity Diagram 23
3.5 Swimlane Diagram 24
3.6 Sequence Diagram 25
3.7 Use Case Diagram 26
3.8 Class Diagram 27
3.9 Safety and Security Requirements 28
3.10 Conclusion 30

4 Architecture 31

4.1 Architectural Model/Style Used 31
4.1.1 Rationale for Choosing the Architectural Model 31
4.2 Technology, Software, and Hardware Used 32

5 Design **33**

5.1 Component Level Design 33

5.1.1 Authentication and User Management 33

5.1.2 Volunteer Management 34

5.1.3 NGO Management 34

5.1.4 Campaign and Event Management 35

5.1.5 Volunteer Matching Component 35

5.1.6 Task and Participation Management 35

5.1.7 Donation and Resource Management 36

5.1.8 Notification and Communication 36

5.1.9 Beneficiary Management 36

5.1.10 Admin Management 37

5.2 Dataflow Diagram 37

5.2.1 Level 0 Dataflow Diagram 38

5.2.2 Level 1 Dataflow Diagram 38

5.3 Entity Relationship Diagram 39

5.4 GUI Design 42

5.4.1 Volunteer Dashboard 42

5.4.2 Volunteer Opportunities Page 43

5.4.3 Active Campaigns and Emergency Relief Page 44

5.4.4 My Assigned Tasks Page 45

5.4.5 Donations and Resources Page 46

5.4.6 Impact and Analytics Page 47

5.4.7 User Profile and Account Menu 48

6 API Documentation **49**

6.1 Introduction 49

6.1.1 API Communication 49

6.1.2 HTTP Methods Used 49

6.2 Module 1: Authentication and User Management 50

6.2.1 User Registration 50
6.2.2 User Login 50
6.2.3 User Profile Management 51
6.3 Module 2: Volunteer Management 51
6.3.1 Create Volunteer Profile 51
6.3.2 View Volunteer Information 52
6.3.3 Update Volunteer Profile 52
6.3.4 Delete Volunteer Profile 52
6.4 Module 3: NGO Management 53
6.4.1 Register a New NGO 53
6.4.2 View NGO Information 53
6.4.3 Update NGO Information 54
6.4.4 Delete NGO Information 54
6.4.5 Verify NGO 54
6.5 Module 4: Campaign and Event Management 55
6.5.1 Create a Campaign 55
6.5.2 View Campaigns 55
6.5.3 Update Campaign 56
6.5.4 Delete Campaign 56
6.6 Module 5: Volunteer Application and Matching 56
6.6.1 Submit Volunteer Application 56
6.6.2 View Applications 57
6.6.3 Update Application Status 57
6.6.4 Volunteer Opportunity Matching 57
6.7 Module 6: Task and Participation Management 58
6.7.1 Create a Task 58
6.7.2 View Assigned Tasks 58
6.7.3 Update Task Status 59
6.7.4 Record Volunteer Attendance 59
6.8 Module 7: Donation and Resource Management 59

6.8.1 Create Donation Record 59
6.8.2 View Donation Records 60
6.8.3 Update Donation Information 60
6.8.4 Resource Management 60
6.9 Module 8: Beneficiary Management 61
6.9.1 Register Beneficiary 61
6.9.2 View Beneficiaries 61
6.9.3 Update Beneficiary Information 61
6.10 Module 9: Notification and Communication 62
6.10.1 Create Notification 62
6.10.2 View Notifications 62
6.10.3 Real-Time Communication 62
6.11 Module 10: Administration and Auditing 63
6.11.1 Manage Users 63
6.11.2 View Audit Logs 63
6.11.3 Generate Reports 63
6.12 API Security 63

7 Testing and Sustainability Plan 65

7.1 Introduction 65
7.2 Testing Strategy 65
7.2.1 Unit Testing 65
7.2.2 Integration Testing 66
7.2.3 Functional Testing 66
7.2.4 Security Testing 66
7.2.5 Usability Testing 67
7.3 Test Cases 67
7.4 Sustainability Plan 68
7.4.1 Technical Sustainability 68
7.4.2 Data Sustainability 68

7.4.3 Operational Sustainability 68
7.4.4 Financial Sustainability 68
7.4.5 Future Development 69
7.5 Conclusion 69

8 CEP Justification **70**
8.1 Introduction 70
8.2 Community Engagement 70
8.3 Social Contribution 70
8.4 Environmental Contribution 71
8.5 Professional and Technical Development 71
8.6 Conclusion 71

9 Conclusion **72**

List of Figures

3.1 Architecture Diagram 22
3.2 State Diagram 23
3.3 Activity Diagram 24
3.4 Swimlane Diagram 25
3.5 Sequence Diagram 26
3.6 Use Case Diagram 27
3.7 Class Diagram 28

5.1 Simplified Entity Relationship Diagram 41
5.2 Volunteer Dashboard Page 43
5.3 Volunteer Opportunities Page 44
5.4 Active Campaigns and Emergency Relief Page 45
5.5 My Assigned Tasks Page 46
5.6 Donations and Resources Page 47
5.7 Impact and Analytics Page 48

List of Tables

2.1 Individual Contribution Matrix 6
2.2 Risk Analysis and Mitigation Strategies 7
2.3 Software Requirements 8
2.4 Project Timeline and Schedule 9
2.5 Estimated Project Budget 9
3.1 Technical Requirements 18
6.1 HTTP Methods Used 50
7.1 Functional Test Cases 67

Executive Summary

The Volunteer & NGO Coordination Platform is a centralized web-based system designed to connect volunteers, Non-Governmental Organizations (NGOs), donors, and beneficiaries through a single digital platform. The main objective of this project is to improve volunteer coordination, NGO activity management, donation transparency, and community support.

The platform provides several important features, including user authentication, volunteer profile management, NGO management, campaign and event management, volunteer opportunity matching, task assignment, donation and resource monitoring, beneficiary management, notification services, and administrative control.

The system helps volunteers discover suitable opportunities based on their skills, interests, location, and availability. NGOs can create campaigns, organize events, assign tasks, manage volunteers, monitor donations, and track resource distribution. Donors can view campaign information and support different social activities, while beneficiaries can receive assistance through NGO campaigns.

The proposed system uses React.js and Tailwind CSS for the frontend, Python FastAPI for backend development, and MySQL as the primary database. SQLAlchemy is used for database interaction, JWT provides authentication, and WebSocket technology supports real-time communication.

Overall, the platform aims to reduce manual coordination, improve organizational efficiency, increase donation transparency, and strengthen volunteer engagement in Bangladesh.

Chapter 1

Introduction

1.1 Goals and Objectives of the Project

The main goal of the Volunteer & NGO Coordination Platform is to establish a centralized digital platform that connects volunteers, NGOs, donors, and beneficiaries.

The specific objectives of the project are:

- To provide a centralized platform for volunteer and NGO coordination.
- To help volunteers discover suitable volunteering opportunities.
- To match volunteers with opportunities based on skills, interests, location, and availability.
- To allow NGOs to manage campaigns, events, tasks, and volunteer applications.
- To maintain accurate records of volunteer participation and attendance.
- To monitor donations and resource distribution.
- To improve communication through notifications and emergency alerts.
- To increase transparency and accountability in NGO activities.
- To provide better support to beneficiaries through organized campaigns.

1.2 Scope of Work

The scope of the project covers the development of a centralized web-based platform for managing volunteering and NGO-related activities. The system includes the following major areas:

User Management: Registration, login, profile management, and role-based access for different users.

Volunteer Management: Volunteer profile creation, skills management, opportunity searching, application submission, and contribution tracking.

NGO Management: NGO registration, profile management, campaign creation, event management, volunteer management, and activity monitoring.

Campaign and Event Management: Creating campaigns, organizing events, managing required volunteers, and tracking campaign progress.

Volunteer Matching: Matching volunteers with suitable opportunities based on skills, interests, location, and availability.

Donation and Resource Management: Maintaining donation records and monitoring the distribution of resources such as food, clothes, medicine, and educational materials.

Beneficiary Management: Maintaining beneficiary information, required assistance, location, and support status.

Notification and Communication: Providing application updates, event reminders, campaign announcements, and emergency notifications.

Administration: Managing users, verifying NGOs, monitoring campaigns, reviewing donations, and generating reports.

1.3 System Overview

The Volunteer & NGO Coordination Platform operates as a centralized web-based system involving five major user groups:

- NGO Administrators
- Volunteers
- Donors
- Beneficiaries
- System Administrators

Each user group has different responsibilities and access permissions. NGO administrators manage organizational activities, while volunteers participate in campaigns and complete assigned tasks. Donors support campaigns through financial or non-financial contributions. Beneficiaries receive necessary assistance, and system administrators monitor the overall platform.

The system follows a client-server architecture. The frontend provides an interactive user interface, the backend processes application requests, and the MySQL database stores system information.

1.4 Structure of the Document

This document is organized into nine chapters.

- **Chapter 1: Introduction** describes the project objectives, scope, and system overview.
- **Chapter 2: Project Management Plan and Feasibility Analysis** explains

team organization, development methodology, risks, resources, timeline, budget, and feasibility.

- **Chapter 3: Software Requirements Specification** describes functional and non-functional requirements, stakeholders, UML diagrams, and security requirements.
- **Chapter 4: Architecture** explains the architectural model and technologies used in the system.
- **Chapter 5: Design** presents component-level design, dataflow diagrams, the entity relationship diagram, and GUI design.
- **Chapter 6: API Documentation** describes the major API operations and their purposes.
- **Chapter 7: Testing and Sustainability Plan** explains testing strategies and future maintenance.
- **Chapter 8: CEP Justification** explains the project's contribution to community, environment, and professional development.
- **Chapter 9: Conclusion** summarizes the project and its expected outcomes.

Chapter 2

Project Management Plan and Feasibility Analysis

2.1 Project Organization

The Volunteer & NGO Coordination Platform was developed as an academic software engineering project by an undergraduate team from the Department of Computer Science and Engineering at the University of Asia Pacific (UAP).

The team followed a collaborative development approach in which each member was assigned specific responsibilities related to project management, development, team coordination, and documentation.

Project Team Roles and Responsibilities

Project Supervisor: Md. Ashraful Alam, Lecturer, Department of Computer Science and Engineering, University of Asia Pacific.

Project Manager: Md. Mahfuzur Rahman [23201206]

- Coordinating overall project activities.
- Managing project tasks and deadlines.
- Monitoring development progress.
- Communicating with team members.
- Ensuring completion of project milestones.

Reporting Lead: Toma Rani Kabiraj [23201214]

- Preparing project documentation.
- Writing project reports and technical descriptions.
- Organizing system requirements and design documentation.
- Preparing diagrams and supporting project presentations.

Team Lead: Wasikul Hasan Fahim [22201037]

- Coordinating technical activities.
- Guiding overall project development.
- Supporting system architecture and design.
- Managing team communication and integration activities.

2.1.1 Individual Contribution to the Project

Each team member contributed to different activities throughout the project lifecycle (Table 2.1).

Table 2.1: Individual Contribution Matrix

Team Member	Requirements	Planning	System Design	Development	Testing	Documentation
Md. Mahfuzur Rahman	Done	Done	Done	Partially	Partially	Partially
Toma Rani Kabiraj	Done	Partially	Done	Partially	Partially	Done
Wasikul Hasan Fahim	Done	Done	Done	Done	Partially	Partially

2.2 Process Model Used

The Agile Software Development Model was selected as the proposed development approach for the Volunteer & NGO Coordination Platform.

Agile is an iterative software development methodology in which a project is divided into smaller development cycles. Each cycle focuses on planning, designing, implementing, testing, and reviewing specific features.

The platform contains several independent modules, such as user management, volunteer management, NGO management, campaign management, donation monitoring, and notification services. Therefore, an iterative approach supports the development and integration of these modules.

The Agile development process includes the following phases:

- **Requirement Gathering:** Identifying system requirements and stakeholder needs.
- **Planning:** Dividing project activities into smaller tasks.
- **Design:** Preparing system architecture, database structure, and user interfaces.
- **Implementation:** Developing frontend, backend, and database functionality.
- **Testing:** Identifying errors and verifying implemented features.
- **Review and Integration:** Reviewing completed modules and integrating them into the main system.

2.2.1 Rationale for Choosing the Lifecycle Model

The Agile model is suitable for this project for the following reasons:

- **Modular Development:** Different system modules can be developed independently.
- **Continuous Improvement:** New requirements and design improvements can be incorporated during development.
- **Early Error Detection:** Testing can be performed after implementing individual features.
- **Team Collaboration:** Team members can coordinate their responsibilities more effectively.
- **Flexible Development:** Changes in volunteer management, NGO activities, and donation features can be handled during development.

2.3 Risk Analysis

Risk analysis helps identify possible problems that may affect the development, security, and operation of the system (Table 2.2).

Table 2.2: Risk Analysis and Mitigation Strategies

Risk Category	Risk Description	Prob.	Impact	Mitigation Strategy
Technical Risk	Server or database failure may interrupt system operations.	Medium	High	Regular backups and proper error handling.
Security Risk	Unauthorized users may access sensitive information.	Medium	High	JWT authentication and role-based access control.
Data Risk	Incorrect donation or resource records may reduce transparency.	Medium	High	Maintain accurate records and audit logs.
Operational Risk	NGOs may provide incomplete campaign information.	Medium	Medium	Validate NGO and campaign information.
Matching Risk	Volunteers may receive unsuitable opportunity recommendations.	Medium	Medium	Improve matching using accurate skills and availability data.
Communication Risk	Excessive notifications may disturb users.	Medium	Medium	Send relevant and prioritized notifications.
Availability Risk	Internet or server failure may interrupt emergency communication.	Medium	High	Backup, recovery, and alternative communication procedures.

2.4 Constraints to Project Implementation

The project may face several implementation constraints:

- Limited development time within the academic semester.
- Limited financial resources for hosting and deployment.
- Dependence on internet connectivity.
- Availability of development hardware and software.
- Complexity of integrating multiple system modules.
- Need for accurate information from NGOs and volunteers.
- Protection of personal and beneficiary-related information.
- Dependence on external services for hosting and real-time communication.

2.5 Hardware and Software Resource Requirements

2.5.1 Software Requirements

The system uses the following technologies:

Table 2.3: Software Requirements

Component	Technology
Frontend Framework	React.js
UI/UX Styling	Tailwind CSS
Backend Framework	Python FastAPI
Programming Language	Python
Database	MySQL
ORM	SQLAlchemy
Authentication	JWT
Real-Time Communication	WebSocket
Version Control	Git and GitHub
Development Environment	Visual Studio Code
Hosting	Hostinger

2.5.2 Hardware Requirements

The minimum hardware requirements for development and usage are:

- Processor: Intel Core i3 or equivalent.
- RAM: Minimum 8 GB.
- Storage: Minimum 256 GB.

- Internet: Stable internet connection.
- Display: Standard laptop or desktop display.

The system can be accessed through modern web browsers on desktop computers, laptops, tablets, and smartphones.

2.6 Project Timeline and Schedule

The project activities are organized into different development phases (Table 2.4).

Table 2.4: Project Timeline and Schedule

Phase	Activities	Expected Deliverables
Phase 1	Project proposal and requirement gathering	Project proposal and SRS
Phase 2	System analysis and design	UML diagrams, architecture, and ERD
Phase 3	Database and backend development	Database structure and backend modules
Phase 4	Frontend development	User interfaces and dashboards
Phase 5	Module integration	Integrated application
Phase 6	Testing and debugging	Tested and improved system
Phase 7	Documentation and presentation	Final report and project demonstration

2.7 Estimated Budget

The estimated budget for developing the academic prototype is presented in Table 2.5.

Table 2.5: Estimated Project Budget

Sl.	Component	Estimated Cost (BDT)
1	Development technologies	0
2	Git and GitHub	0
3	Database software	0
4	Documentation	500
5	Internet and communication	500
6	Hosting and deployment	1,500
	Total	**2,500**

The budget is an estimated academic prototype budget. Actual deployment expenses may vary depending on hosting, domain, storage, and external service requirements.

2.8 Social, Cultural, and Environmental Impact of the Project

2.8.1 Social Impact

The platform is expected to create positive social impacts in the following ways:

- **Volunteer Engagement:** Helps volunteers find suitable social activities.
- **Community Support:** Improves coordination of relief campaigns and community assistance.
- **Donation Transparency:** Helps NGOs maintain organized donation and resource records.
- **Beneficiary Support:** Helps identify people who need assistance.
- **NGO Collaboration:** Creates a centralized environment for NGO activities.

2.8.2 Cultural Impact

- Encourages a culture of volunteering and social responsibility.
- Promotes cooperation among individuals and organizations.
- Encourages young people to participate in community development.
- Supports teamwork and humanitarian activities.

2.8.3 Environmental Impact

- Reduces paper-based documentation.
- Supports digital campaign and volunteer records.
- Reduces unnecessary physical communication.
- Helps coordinate environmental campaigns such as tree plantation and clean-up activities.

2.9 Feasibility Analysis

A feasibility analysis is conducted to evaluate whether the proposed system can be developed and maintained successfully. The project is evaluated using the TELOS framework, which includes Technical, Economic, Legal, Operational, and Schedule Feasibility.

2.9.1 Technical Feasibility

The project is technically feasible because it uses widely available technologies such as React.js, Python FastAPI, MySQL, and SQLAlchemy. These technologies provide the necessary facilities for frontend development, backend processing, database management, authentication, and communication.

2.9.2 Economic Feasibility

The system is economically feasible as most development technologies are open-source or available through free academic services. The academic prototype requires limited financial investment. However, production deployment may require additional costs for hosting, domain registration, storage, and maintenance.

2.9.3 Operational Feasibility

The platform is operationally feasible because it provides separate functionalities for NGO administrators, volunteers, donors, beneficiaries, and system administrators. A simple and user-friendly interface can help users perform their activities without requiring advanced technical knowledge.

2.9.4 Legal and Ethical Feasibility

The system is designed to support responsible data management and secure access to user information. Personal information, donor records, and beneficiary details should only be accessible to authorized users. The platform should follow applicable data protection, privacy, donation transparency, and ethical practices.

2.9.5 Schedule Feasibility

The project is planned within the academic semester. The development activities are divided into requirement analysis, system design, implementation, testing, and documentation. Parallel work among team members can help complete the required modules within the available academic schedule.

Therefore, the project is considered feasible within the proposed academic scope.

Chapter 3

Requirement Specifications

3.1 Software Requirements Specification (SRS)

3.1.1 Introduction

Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of the proposed Volunteer & NGO Coordination Platform. The system is designed as a centralized web and mobile-based platform that connects volunteers, NGOs, donors, and beneficiaries.

The main purpose of the platform is to improve volunteer opportunity discovery, NGO activity coordination, donation and resource monitoring, communication, and accountability.

Scope

The platform will provide a centralized environment where volunteers can discover suitable opportunities, apply for activities, participate in events, complete assigned tasks, and track their contributions.

NGOs will be able to manage campaigns, events, tasks, volunteers, attendance, donations, and resources. Donors will be able to view campaign information and donation-related information, while beneficiaries will receive support through NGO campaigns and resource distribution activities.

The system will also provide volunteer matching based on skills, interests, location, and availability. Real-time notifications and communication will be supported for event updates, campaign announcements, and emergency situations.

3.1.2 Overall Description

Product Perspective

The Volunteer & NGO Coordination Platform will operate as a centralized system connecting four major stakeholder groups:

- NGO Administrators
- Volunteers
- Donors
- Beneficiaries

The platform will integrate volunteer management, opportunity management, task and event coordination, contribution tracking, donation monitoring, resource monitoring, authentication, notifications, and auditing within a single system.

The frontend will be developed using React.js and Tailwind CSS. The backend will use Python FastAPI, while MySQL will be used as the primary database. SQLAlchemy will provide database interaction, JWT will be used for authentication, and WebSocket technology will support real-time communication.

Stakeholders and User Classes

NGO Administrators. NGO administrators will be responsible for managing NGO-related activities. They will be able to:

- Create and manage campaigns.
- Create and manage volunteer opportunities.
- Manage events and tasks.
- Review volunteer applications.
- Assign tasks to volunteers.
- Record volunteer attendance.
- Monitor donations and resources.
- Review volunteer contributions.

Volunteers. Volunteers will be able to:

- Create and manage their profiles.
- Browse available opportunities.
- Search for suitable opportunities.
- Apply for volunteer activities.

- Participate in events.
- View assigned tasks.
- Complete assigned tasks.
- Track attendance and contributions.

Donors. Donors will be able to:

- View NGO campaigns.
- Access campaign-related information.
- Support donation activities.
- View relevant donation and resource utilization information.

Beneficiaries. Beneficiaries will receive aid and support through NGO campaigns and resource distribution activities.

Operating Environment

The system will operate as a web and mobile-based platform. The main technologies will include:

- Frontend: React.js and Tailwind CSS
- Backend: Python FastAPI
- Database: MySQL
- ORM: SQLAlchemy
- Authentication: JWT
- Real-Time Communication: WebSocket
- Version Control: Git and GitHub
- Hosting: Hostinger

3.1.3 Functional Requirements

User Authentication

The system shall:

- Provide secure user registration and login.
- Support role-based access for NGO administrators, volunteers, donors, and beneficiaries.
- Verify user credentials before granting access.

- Prevent unauthorized access to restricted resources.
- Use JWT-based authentication for secure access management.

Volunteer Opportunity Management

The system shall:

- Allow NGO administrators to create volunteer opportunities.
- Allow administrators to view existing opportunities.
- Allow administrators to update opportunity information.
- Allow administrators to remove or deactivate opportunities.
- Store information such as required skills, interests, location, availability, event details, and task requirements.
- Allow volunteers to browse available opportunities.

Volunteer Matching

The system shall:

- Match volunteers with suitable opportunities.
- Consider volunteer skills, interests, location, and availability.
- Rank suitable opportunities according to matching relevance.
- Support similarity scoring and ranking logic.
- Allow the matching mechanism to be tested and improved using system data.

Application and Task Management

The system shall:

- Allow volunteers to apply for available opportunities.
- Allow NGO administrators to view volunteer applications.
- Allow administrators to accept or reject applications.
- Allow administrators to assign tasks to volunteers.
- Allow volunteers to view assigned tasks.
- Allow volunteers to track task status.
- Maintain records of applications and assigned tasks.

Event and Campaign Management

The system shall:

- Allow NGO administrators to create and manage events.
- Allow administrators to create and manage campaigns.
- Allow administrators to assign volunteers to events.
- Maintain event information and participation records.
- Support attendance tracking.
- Allow volunteers to view upcoming events.
- Associate relevant events, tasks, donations, and resources with campaigns where applicable.

Contribution Tracking

The system shall:

- Maintain records of volunteer participation.
- Track attendance for events and activities.
- Maintain volunteer contribution records.
- Allow volunteers to view their participation history.
- Allow volunteers to view their contribution history.
- Allow NGO administrators to review volunteer participation and contribution information.

Donation and Resource Monitoring

The system shall:

- Maintain donation records associated with campaigns or activities.
- Allow donors to access relevant campaign information.
- Allow NGO administrators to monitor donation records.
- Support resource distribution monitoring.
- Maintain structured records of resource allocation and usage.
- Allow authorized users to review resource utilization information.

Notification and Real-Time Communication

The system shall:

- Provide notifications for relevant event and campaign updates.
- Provide emergency alerts when necessary.
- Notify volunteers about applications, tasks, events, and participation.
- Allow NGO administrators to communicate important updates to relevant users.

- Use WebSocket technology for real-time notification delivery.
- Minimize unnecessary notifications to reduce notification fatigue.

Performance Analysis

The system shall:

- Allow NGO administrators to review volunteer participation.
- Provide performance-related information based on participation and contribution data.
- Support analysis of volunteer activity and engagement.
- Support reporting and ranking logic where appropriate.

Auditing

The system shall:

- Maintain records of important administrative actions.
- Maintain records of important resource-related actions.
- Support investigation of donation and resource tracking activities.
- Restrict audit information to authorized users.
- Maintain sufficient information for accountability and traceability.

3.1.4 Non-Functional Requirements

Security

The system shall:

- Provide secure authentication and authorization.
- Implement role-based access control.
- Protect volunteer, donor, beneficiary, and NGO information.
- Prevent unnecessary exposure of sensitive information.
- Record important administrative and resource-management actions through audit logs.

Performance

The system should:

- Respond efficiently to normal user requests.
- Provide opportunity search and matching without unnecessary delay.
- Deliver real-time notifications with minimal delay.
- Support multiple users and increasing amounts of system data.

Usability

The system should:

- Provide a simple and user-friendly interface.
- Clearly display opportunities, events, campaigns, tasks, and contribution information.
- Provide role-specific functions without unnecessary complexity.
- Provide clear form validation and error messages.

Reliability and Responsiveness

The system should:

- Maintain consistent records of participation, donations, resources, events, and tasks.
- Handle errors without corrupting important records.
- Provide backup and recovery for critical data.
- Provide a responsive interface across supported devices.
- Maintain consistent real-time functionality across supported clients.

3.1.5 Technical Requirements

The technical requirements of the system are shown in Table 3.1.

Table 3.1: Technical Requirements

Component	**Technology**
Frontend	React.js
UI/UX	Tailwind CSS
Backend	Python FastAPI
Database	MySQL
ORM	SQLAlchemy
Authentication	JWT
Real-Time Communication	WebSocket
Version Control	Git and GitHub
Hosting	Hostinger

3.1.6 System Constraints

The system will have the following constraints:

- React.js and Tailwind CSS must be used for the frontend.
- Python FastAPI must be used for backend development.
- MySQL must be used as the primary relational database.
- SQLAlchemy must be used for database interaction.
- JWT must be used for authentication.
- WebSocket technology must be used for real-time communication.
- The system requires internet connectivity for online access and real-time communication.

3.1.7 Assumptions and Dependencies

The system is based on the following assumptions:

- Volunteers will provide accurate information about their skills, interests, location, and availability.
- NGOs will provide accurate information about opportunities, events, campaigns, tasks, donations, and resources.
- Users will use the platform according to their assigned roles.
- NGOs will maintain updated opportunity and campaign information.
- The accuracy of volunteer matching depends on the quality and completeness of available data.
- Donation and resource transparency depends on NGOs maintaining accurate records.

The system depends on internet connectivity, the availability of the backend server, database services, and real-time communication services.

3.1.8 Proposed Solution

The proposed solution is a centralized web and mobile-based platform that connects volunteers, NGOs, donors, and beneficiaries through a single system.

Centralized Platform. The platform will provide a common environment where all major stakeholder groups can access services according to their roles. This will reduce fragmented coordination and improve communication.

NGO Program Management. NGO administrators will be able to manage campaigns, events, tasks, volunteers, and attendance through the platform. This will help NGOs coordinate activities and maintain organized participation records.

Volunteer Matching. The platform will provide a matching mechanism that helps volunteers discover suitable opportunities based on skills, interests, location, and availability. The matching mechanism will use similarity scoring and ranking logic.

Donation and Resource Tracking. The platform will maintain donation records and monitor resource distribution. Structured records and auditing will support transparency, accountability, and resource utilization monitoring.

Real-Time Communication. The platform will provide event updates, campaign notifications, and emergency alerts. WebSocket-based communication will support real-time delivery of relevant notifications.

3.1.9 Expected System Output

The expected outputs of the proposed system are:

- A centralized volunteer and NGO coordination platform.
- Smart volunteer-to-opportunity matching.
- Efficient NGO event, campaign, task, and volunteer management.
- Donation and resource monitoring.
- Real-time notifications and communication.
- Volunteer performance and contribution tracking.
- Audit records supporting accountability.

3.1.10 Social Impact

The proposed platform is expected to create the following social benefits:

Stronger Volunteer Engagement. Volunteers can discover opportunities that better match their skills, interests, location, and availability.

Better Community Support. NGOs can coordinate volunteers, events, relief campaigns, and aid delivery more efficiently.

Greater Transparency. Donation and resource tracking can improve accountability and provide better visibility into resource utilization.

Improved NGO Coordination. Centralized management can reduce fragmented volunteer and campaign coordination.

3.1.11 Risk Analysis

Inaccurate Volunteer Opportunity Matching. The matching system may recommend unsuitable opportunities if volunteer or opportunity information is incomplete or inaccurate.
Mitigation: The matching mechanism should be tested and improved using skills, interests, location, availability, and other relevant attributes.

Notification Overload. Excessive notifications may cause users to ignore important alerts.
Mitigation: The system should provide relevant and prioritized notifications and avoid unnecessary alerts.

Privacy of User Data. Volunteer, donor, and beneficiary information may be exposed if access controls are insufficient.
Mitigation: Role-based access control should be implemented and unnecessary data exposure should be minimized.

Donation and Resource Tracking Errors. Incorrect or incomplete records may reduce transparency and accountability.
Mitigation: Structured records and audit logs should be maintained for donation and resource-related activities.

System Failure During Emergency Campaigns. System failures during emergency campaigns may disrupt communication and coordination.
Mitigation: The system should use appropriate error handling, backups, reliability measures, and recovery mechanisms.

3.2 Architecture Diagram

The architecture diagram shows the main components of the Volunteer Management System and how the frontend, backend, authentication, communication, and database

components interact.

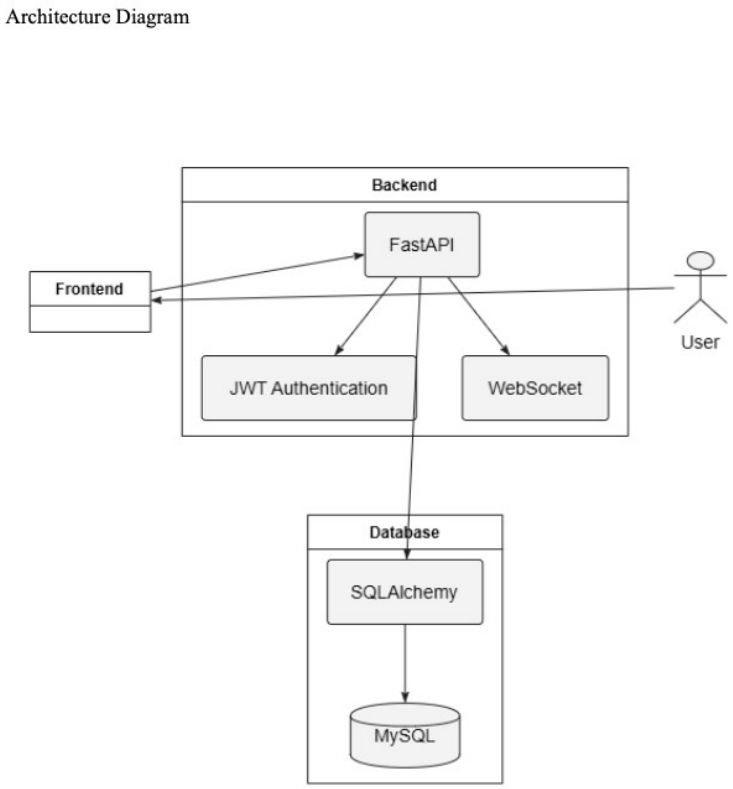


Figure 3.1: Architecture Diagram

Main Components: FastAPI handles backend services, JWT provides authentication, WebSocket enables real-time communication, and SQLAlchemy connects the application with the MySQL database.

3.3 State Diagram

The state diagram represents the different states of a volunteer application and task throughout the volunteer management process.

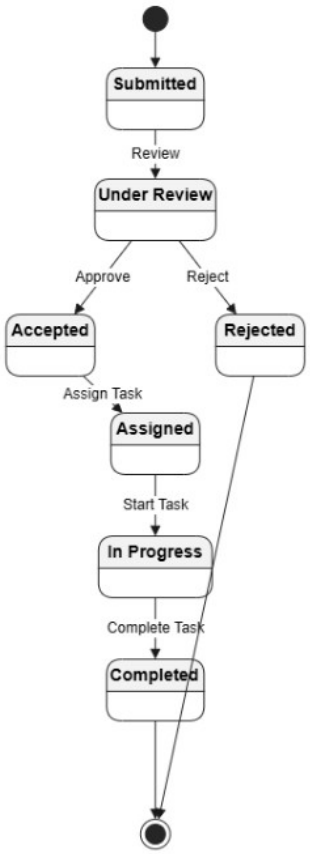


Figure 3.2: State Diagram

The application moves from *Submitted* to *Under Review*, and can be either *Accepted* or *Rejected*. Accepted applications proceed through task assignment, progress, and completion.

3.4 Activity Diagram

The activity diagram illustrates the overall workflow followed by a volunteer when applying for an opportunity.

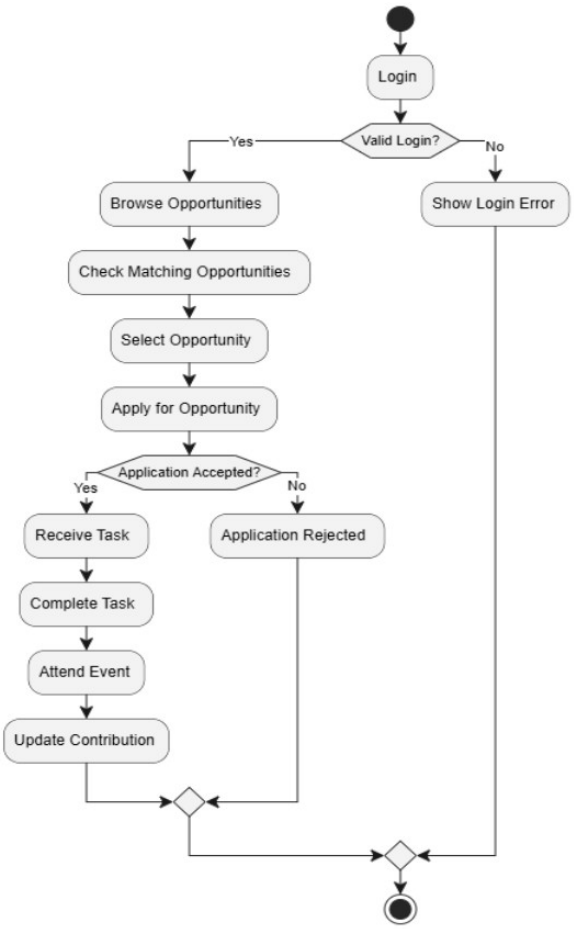


Figure 3.3: Activity Diagram

The process includes login, browsing and selecting opportunities, applying, receiving a task, completing the task, and updating the contribution.

3.5 Swimlane Diagram

The swimlane diagram represents the sequence of activities involved in volunteering, including login, opportunity selection, application, task completion, and contribution updates.

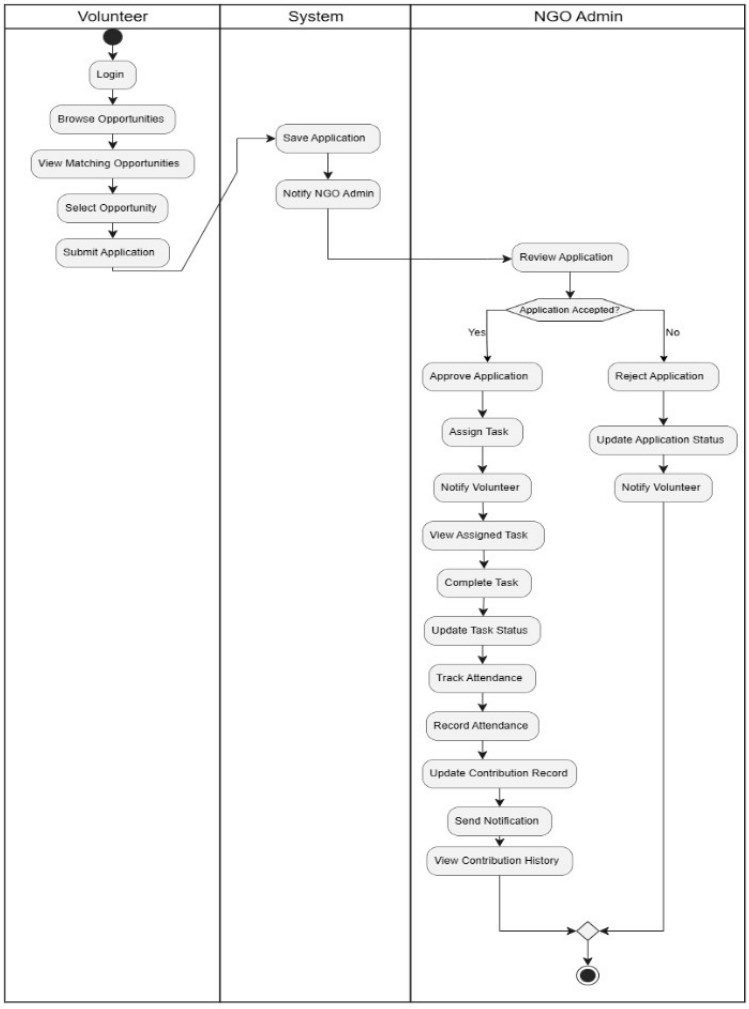


Figure 3.4: Swimlane Diagram

The main activities include browsing opportunities, applying, receiving tasks, completing tasks, attending events, and updating contributions.

3.6 Sequence Diagram

The sequence diagram shows the interaction between the Volunteer, System, and NGO Admin during the volunteer application and task management process.

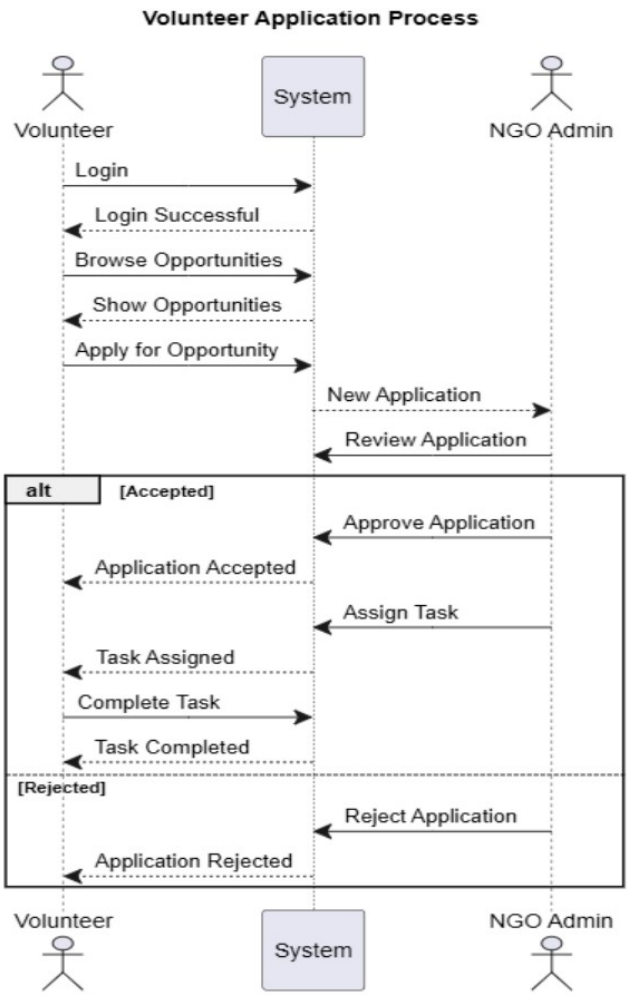


Figure 3.5: Sequence Diagram

Entities: The Volunteer submits applications and completes tasks. The System manages applications and notifications. The NGO Admin reviews applications, assigns tasks, and records attendance and contributions.

3.7 Use Case Diagram

The use case diagram shows the major interactions between the system and its primary actors during the volunteer application process.

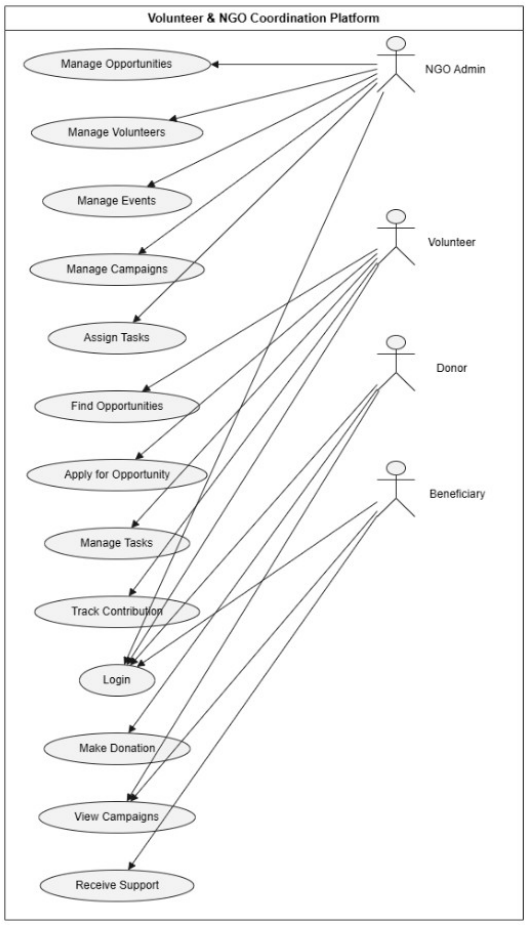


Figure 3.6: Use Case Diagram

Actors: The main actors are the Volunteer and NGO Admin. The Volunteer can log in, browse opportunities, apply, and complete assigned tasks, while the NGO Admin reviews and manages applications.

3.8 Class Diagram

The class diagram represents the main entities of the Volunteer Management System and their relationships.

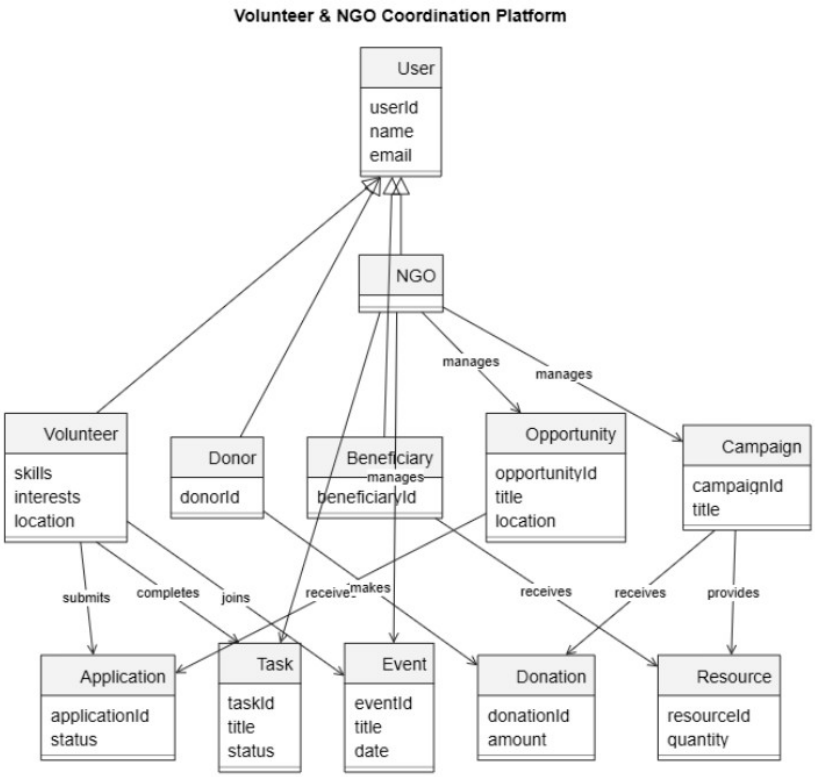


Figure 3.7: Class Diagram

Main Entities: The system includes User, NGO, Volunteer, Donor, Beneficiary, Opportunity, Campaign, Application, Task, Event, Donation, and Resource entities. These entities represent users, volunteering activities, donations, campaigns, and related system records.

3.9 Safety and Security Requirements

Authentication and Access Control

- All users must authenticate via JWT-based login before accessing any protected resource.
- Passwords must be stored using a strong one-way hash (e.g., bcrypt or argon2), never in plain text.
- The system must enforce role-based access control (RBAC) for the four roles: NGO Admin, Volunteer, Donor, and Beneficiary.
- JWT tokens must have a limited expiry time and support refresh-token rotation.
- Failed login attempts should be rate-limited to prevent brute-force attacks.

Data Protection

- Sensitive personal data (volunteer contact information, donor payment details, beneficiary records) must be encrypted at rest in the MySQL database and encrypted in transit using HTTPS/TLS for all API and WebSocket traffic.
- Donor payment information should follow standard data-protection practices, avoiding direct storage of raw payment data.
- Personally Identifiable Information (PII) should be masked or access-restricted in dashboards unless required for a specific task.

Input Validation and Application Security

- All API endpoints developed with FastAPI must validate and sanitize input to prevent SQL injection, XSS, and CSRF attacks.
- File uploads, if supported, must be validated for type and size before storage.
- SQLAlchemy ORM usage should rely on parameterized queries rather than raw SQL statements.

Auditing and Accountability

- All administrative actions, such as approving or rejecting applications and editing donation or resource records, must be logged with timestamp, user ID, and action type.
- Audit logs must be immutable and accessible only to authorized NGO Admins or system auditors.

System and Network Security

- WebSocket connections must be authenticated to prevent unauthorized notification injection.
- The system must implement HTTPS across the web and mobile platforms with valid SSL/TLS certificates.
- Backend dependencies (FastAPI, SQLAlchemy, etc.) should receive regular security updates and patches.

Safety and Operational Reliability

- Critical records such as donations, tasks, and attendance must be backed up regularly with a documented recovery process.

- Core application and task workflows should continue functioning even if the matching or notification service fails.
- Beneficiary data should be handled with additional sensitivity, minimizing exposure to volunteers or donors who do not require access.
- A basic incident response process should be defined for identifying, reporting, and resolving security incidents, including breach notification where applicable.

3.10 Conclusion

The Volunteer & NGO Coordination Platform is intended to bring volunteers, NGOs, donors, and beneficiaries together in one centralized system. The platform will support volunteer opportunity discovery and matching, NGO event and campaign management, task assignment, attendance and contribution tracking, donation and resource monitoring, real-time notifications, and auditing.

By integrating these functions into one platform, the system aims to improve volunteer engagement, NGO coordination, donation transparency, donor trust, and community support. The requirements defined in this chapter provide the foundation for subsequent system design, implementation, testing, and evaluation.

Chapter 4

Architecture

4.1 Architectural Model/Style Used

The Volunteer & NGO Coordination Platform follows a client-server architecture with separate frontend, backend, and database components.

The frontend provides the graphical user interface through which users interact with the platform. The backend processes user requests, performs business operations, validates information, and communicates with the database.

The MySQL database stores user profiles, NGO information, campaigns, events, applications, tasks, donations, resources, and other system records.

The architecture consists of the following layers:

1. **Presentation Layer.** The presentation layer is developed using React.js and Tailwind CSS. It provides dashboards, registration forms, campaign pages, volunteer profiles, and other user interfaces.
2. **Application Layer.** The application layer uses Python FastAPI to handle business logic, API requests, authentication, volunteer matching, and system operations.
3. **Data Access Layer.** SQLAlchemy is used to communicate with the database and manage database operations.
4. **Database Layer.** MySQL stores and manages the structured information required by the system.

JWT provides authentication, while WebSocket supports real-time communication.

4.1.1 Rationale for Choosing the Architectural Model

The client-server architectural model is suitable for the platform for the following reasons:

- Separation of frontend and backend responsibilities.
- Centralized data management.
- Easier maintenance and development.
- Better organization of system components.

- Support for multiple user roles.
- Improved integration between different modules.
- Possibility of future scalability.

4.2 Technology, Software, and Hardware Used

The project uses React.js, Tailwind CSS, Python FastAPI, MySQL, SQLAlchemy, JWT, WebSocket, Git, and GitHub. These technologies provide the required facilities for interface development, backend processing, database management, authentication, communication, and version control.

The system can be accessed through standard computers and mobile devices using modern web browsers.

Chapter 5

Design

5.1 Component Level Design

The Volunteer & NGO Coordination Platform is designed as a web-based system that connects volunteers, NGOs, donors, and beneficiaries through a centralized platform. The system is divided into several major components to make the system organized, scalable, secure, and easy to maintain.

The major components of the system are:

- Authentication and User Management Component
- Volunteer Management Component
- NGO Management Component
- Campaign and Event Management Component
- Volunteer Matching Component
- Task and Participation Management Component
- Donation and Resource Management Component
- Notification and Communication Component
- Beneficiary Management Component
- Admin Management Component

5.1.1 Authentication and User Management

This component manages user registration, login, logout, profile management, and role-based access.

Main functions:

- User registration
- Secure login and logout
- Password management
- Profile update
- Role management

- Account verification

The system provides different access levels for Volunteers, NGOs, Donors, Beneficiaries, and Admins.

5.1.2 Volunteer Management

The Volunteer Management component allows volunteers to create and maintain their profiles. Volunteer information includes:

- Name
- Email
- Phone number
- Skills
- Interests
- Location
- Availability
- Previous experience

Volunteers can search for suitable opportunities, apply for events, accept tasks, and track their participation.

5.1.3 NGO Management

This component allows NGOs to manage their organizational information and activities.

Main functions:

- NGO registration
- NGO profile management
- Create campaigns
- Create volunteer opportunities
- Create events
- Assign tasks
- Manage volunteers
- Monitor participation

NGOs can also review volunteer applications and communicate with selected volunteers.

5.1.4 Campaign and Event Management

The Campaign and Event Management component allows NGOs to create and manage social campaigns and events. An NGO can provide:

- Campaign title
- Description
- Location
- Date and time
- Required volunteers
- Required resources
- Campaign status

Volunteers can view available campaigns and apply according to their interests, skills, and availability.

5.1.5 Volunteer Matching Component

The Volunteer Matching component helps connect suitable volunteers with appropriate NGO activities. The matching process considers:

- Volunteer skills
- Volunteer interests
- Location
- Availability
- Required skills
- Activity type

This component helps reduce manual searching and improves coordination between NGOs and volunteers.

5.1.6 Task and Participation Management

This component manages tasks assigned to volunteers.

Main functions:

- Create tasks
- Assign tasks
- Accept or reject tasks
- Update task status

- Track participation
- Record completed activities

Task status may include: *Pending* → *Accepted* → *In Progress* → *Completed*.

5.1.7 Donation and Resource Management

The Donation and Resource Management component manages financial and non-financial contributions. Donors can provide:

- Monetary donations
- Food
- Clothes
- Medicine
- Educational materials
- Other necessary resources

NGOs can create donation requests and maintain donation records.

5.1.8 Notification and Communication

This component provides communication between users and the system. The system can send notifications for:

- New volunteer opportunities
- Application status
- Task assignments
- Event reminders
- Campaign updates
- Donation updates
- Emergency announcements

A communication system may also allow volunteers and NGOs to exchange messages.

5.1.9 Beneficiary Management

The Beneficiary Management component helps NGOs maintain information about people or communities who need support. The system can store:

- Beneficiary information
- Required assistance

- Location
- Campaign information
- Support status

This helps NGOs identify and provide appropriate assistance to beneficiaries.

5.1.10 Admin Management

The Admin component is responsible for monitoring and controlling the whole platform. Admin functions include:

- Manage users
- Verify NGOs
- Monitor campaigns
- Manage reported content
- Monitor donations
- Manage system records
- Generate reports
- Maintain system security

5.2 Dataflow Diagram

The Dataflow Diagram (DFD) represents how data moves between different users, processes, and databases of the Volunteer & NGO Coordination Platform. The main external entities of the system are:

- Volunteer
- NGO
- Donor
- Beneficiary
- Admin

The main processes are:

- User Authentication and Profile Management
- Volunteer and NGO Management
- Campaign and Event Management
- Volunteer Matching and Application
- Task and Participation Management
- Donation and Resource Management

- Notification and Communication
- Administration

5.2.1 Level 0 Dataflow Diagram

At Level 0, the complete platform is represented as one major process.

External Entities → System → External Entities

- Volunteer → Volunteer & NGO Coordination Platform
- NGO → Volunteer & NGO Coordination Platform
- Donor → Volunteer & NGO Coordination Platform
- Beneficiary → Volunteer & NGO Coordination Platform
- Admin → Volunteer & NGO Coordination Platform

The system receives information from these entities and provides appropriate services and responses.

5.2.2 Level 1 Dataflow Diagram

The main system can be divided into the following processes:

Process 1: Authentication & User Management *Volunteer/NGO/Donor/Beneficiary → Registration/Login → Authentication System → User Data*

Process 2: Campaign & Event Management *NGO → Campaign/Event Information → Campaign Management → Campaign Database*

Process 3: Volunteer Application & Matching *Volunteer → Skills/Interest/Availability → Matching System*
Matching System → Suitable Opportunities → Volunteer

Process 4: Task & Participation Management *NGO → Task Assignment → Task Management → Volunteer*
Volunteer → Task Status → Task Management

Process 5: Donation & Resource Management *Donor → Donation/Resource Information → Donation Management → NGO*

Process 6: Notification & Communication *System → Notifications → Volunteer/NGO/Donor/Beneficiary*

Process 7: Administration *Admin → User/Campaign/Donation Management → Admin Panel → System Database*

Main Data Stores

The system maintains several databases/data stores:

- D1: User Database
- D2: Volunteer Database
- D3: NGO Database
- D4: Campaign/Event Database
- D5: Application Database
- D6: Task/Participation Database
- D7: Donation Database
- D8: Notification Database
- D9: Beneficiary Database

5.3 Entity Relationship Diagram

The Entity Relationship Diagram (ERD) represents the major entities of the Volunteer & NGO Coordination Platform and the relationships between them.

Main Entities

1. **User**
   Attributes: User_ID (PK), Name, Email, Password, Phone, Role, Location.
   A User can have a Volunteer, NGO, Donor, or Beneficiary profile.
2. **Volunteer**
   Attributes: Volunteer_ID (PK), User_ID (FK), Skills, Interests, Availability, Experience.
   A volunteer can apply for multiple opportunities and participate in multiple tasks.
3. **NGO**
   Attributes: NGO_ID (PK), User_ID (FK), NGO_Name, Description, Address, Contact, Verification_Status.
   An NGO can create multiple campaigns, events, and tasks.

4. **Campaign**
   Attributes: Campaign_ID (PK), NGO_ID (FK), Title, Description, Location, Start_Date, End_Date, Status.
   One NGO can create many campaigns.
5. **Event**
   Attributes: Event_ID (PK), Campaign_ID (FK), Event_Name, Date, Time, Location, Description.
   A campaign can contain multiple events.
6. **Application**
   Attributes: Application_ID (PK), Volunteer_ID (FK), Campaign_ID (FK), Application_Date, Status.
   A volunteer can submit multiple applications, and a campaign can receive applications from multiple volunteers.
7. **Task**
   Attributes: Task_ID (PK), Campaign_ID (FK), Volunteer_ID (FK), Task_Name, Description, Deadline, Status.
   Tasks can be assigned to volunteers for a specific campaign.
8. **Donation**
   Attributes: Donation_ID (PK), Donor_ID (FK), NGO_ID (FK), Amount, Donation_Type, Donation_Date, Status.
   A donor can make multiple donations to NGOs.
9. **Beneficiary**
   Attributes: Beneficiary_ID (PK), Name, Contact, Location, Required_Assistance, Status.
   Beneficiaries can receive support through NGO campaigns.
10. **Notification**
    Attributes: Notification_ID (PK), User_ID (FK), Message, Notification_Type, Date, Status.
    A user can receive multiple notifications.
11. **Participation**
    Attributes: Participation_ID (PK), Volunteer_ID (FK), Event_ID (FK), Attendance, Participation_Date, Hours.
    This entity stores volunteer participation and attendance information.

Main Relationships

- **User – Volunteer:** A user can have a volunteer profile.
- **User – NGO:** A user can manage an NGO profile.
- **NGO – Campaign:** One NGO can create many campaigns.
- **Campaign – Event:** One campaign can contain many events.

- **Volunteer – Application:** One volunteer can submit many applications.
- **Campaign – Application:** One campaign can receive many applications.
- **Campaign – Task:** One campaign can have many tasks.
- **Volunteer – Task:** A volunteer can receive multiple tasks.
- **Donor – Donation:** One donor can make many donations.
- **NGO – Donation:** An NGO can receive many donations.
- **Volunteer – Participation:** A volunteer can participate in many events.
- **Event – Participation:** An event can have many participants.
- **User – Notification:** One user can receive many notifications.
- **Campaign – Beneficiary:** Campaigns can provide support to beneficiaries.

Simplified ERD Structure

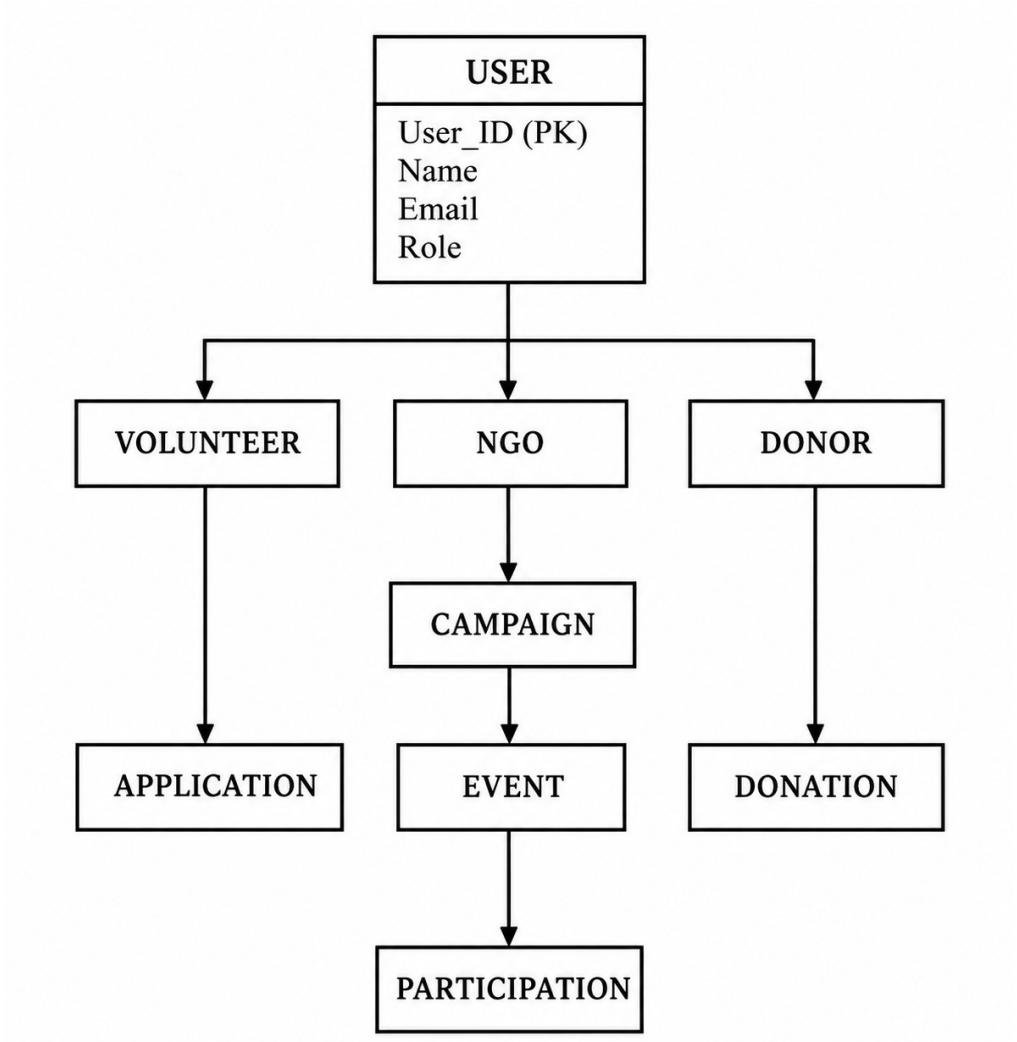


Figure 5.1: Simplified Entity Relationship Diagram

5.4 GUI Design

The Graphical User Interface (GUI) of the Volunteer & NGO Coordination Platform is designed to provide a simple, attractive, and user-friendly experience for all users.

The interface focuses on easy navigation, clear information presentation, and role-based functionality. Users can access different system features through a structured dashboard and navigation menu.

The GUI design includes volunteer dashboards, opportunity listings, campaign information, assigned tasks, donation records, and performance analytics. The following sections describe the major interface pages of the system.

5.4.1 Volunteer Dashboard

The Volunteer Dashboard provides an overview of the volunteer's activities and contributions. The dashboard displays:

- Total volunteer hours.
- Number of events attended.
- Assigned tasks.
- Donation contribution information.
- Smart volunteer matching recommendations.
- Upcoming activities and emergency announcements.

This page helps volunteers monitor their activities and quickly access important features.

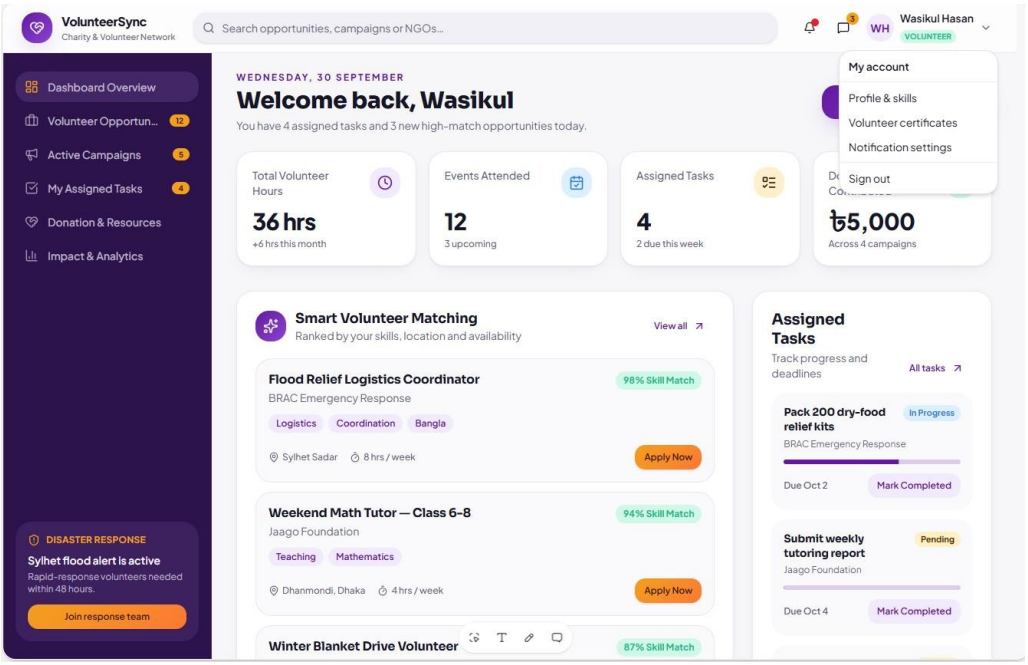


Figure 5.2: Volunteer Dashboard Page

5.4.2 Volunteer Opportunities Page

The Volunteer Opportunities page displays available volunteering activities from different NGOs. The page provides:

- Opportunity titles and descriptions.
- NGO or organization names.
- Required skills.
- Location information.
- Time requirements.
- Volunteer matching percentage.
- Application options.

Volunteers can explore opportunities and apply for activities according to their skills, interests, and availability.

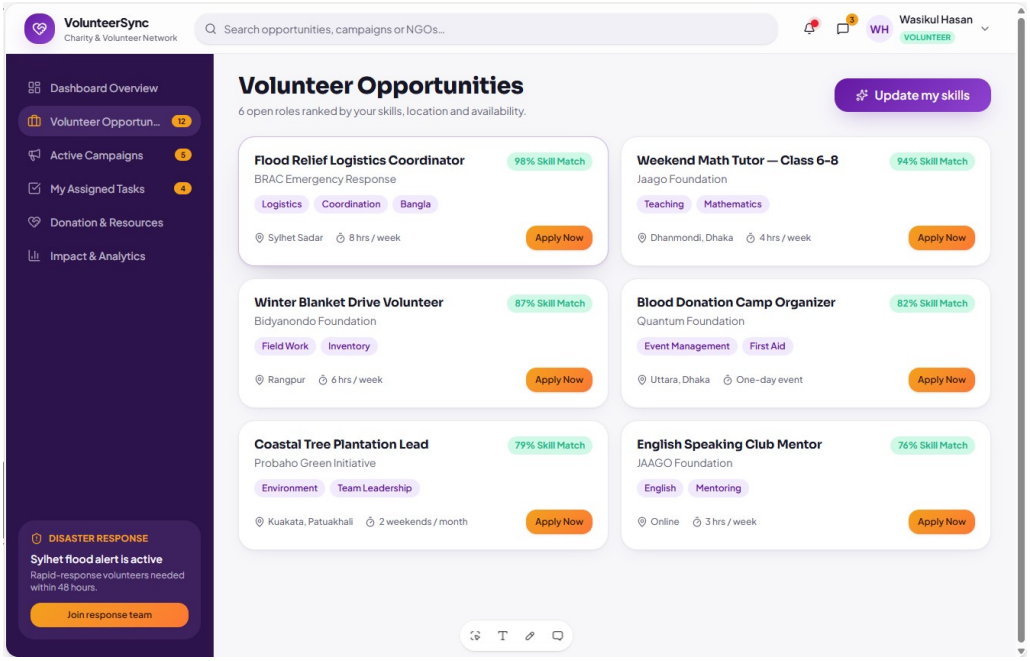


Figure 5.3: Volunteer Opportunities Page

5.4.3 Active Campaigns and Emergency Relief Page

This page displays ongoing campaigns and emergency relief activities organized by different NGOs. The interface includes:

- Campaign names.
- Organization information.
- Campaign descriptions.
- Donation progress.
- Required volunteer information.
- Emergency campaign indicators.
- Donation and volunteer participation options.

This page allows users to identify active campaigns and participate in social activities.

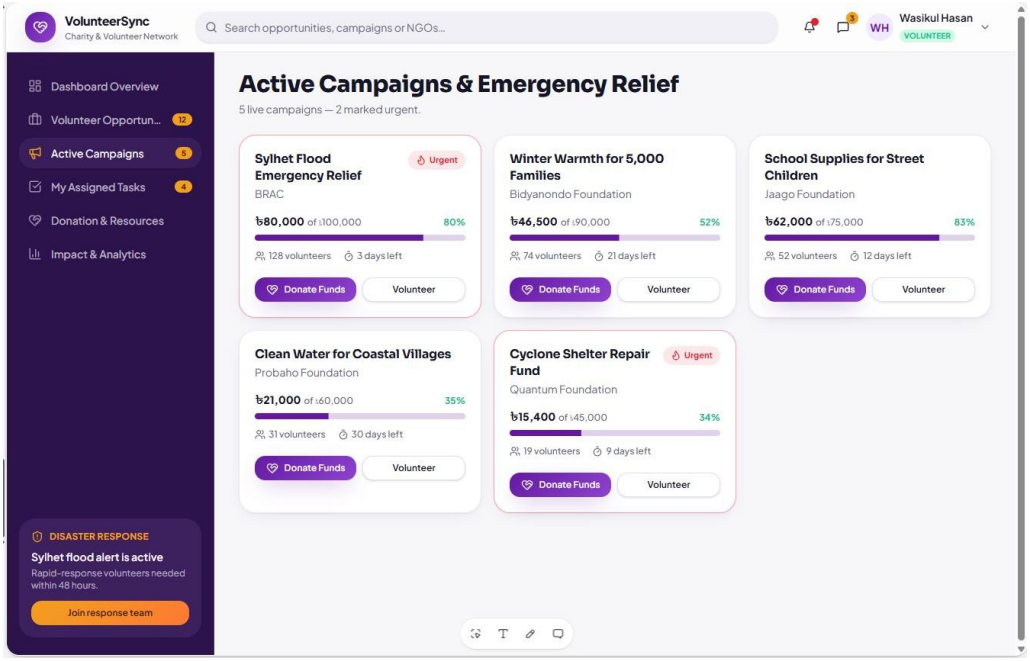


Figure 5.4: Active Campaigns and Emergency Relief Page

5.4.4 My Assigned Tasks Page

The My Assigned Tasks page allows volunteers to view and manage their assigned responsibilities. The page displays:

- Assigned task names.
- Related campaign information.
- Task status.
- Completion progress.
- Deadlines.
- Task completion options.

The task status may include Pending, In Progress, and Completed. This feature helps volunteers monitor their responsibilities and complete assigned activities within the required deadlines.

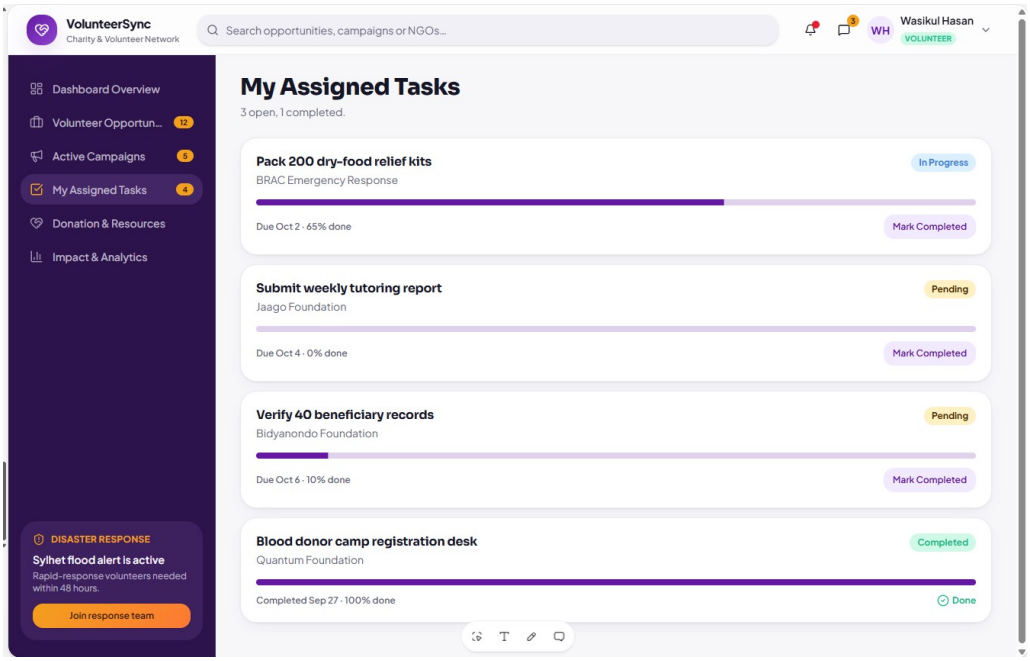


Figure 5.5: My Assigned Tasks Page

5.4.5 Donations and Resources Page

The Donations and Resources page provides information about donation activities and resource contributions. The interface displays:

- Total contributed amount.
- Campaign names.
- Organization names.
- Donation amounts.
- Payment methods.
- Donation dates.
- Donation history.

This page helps users review their donation history and understand how their contributions are associated with different campaigns.

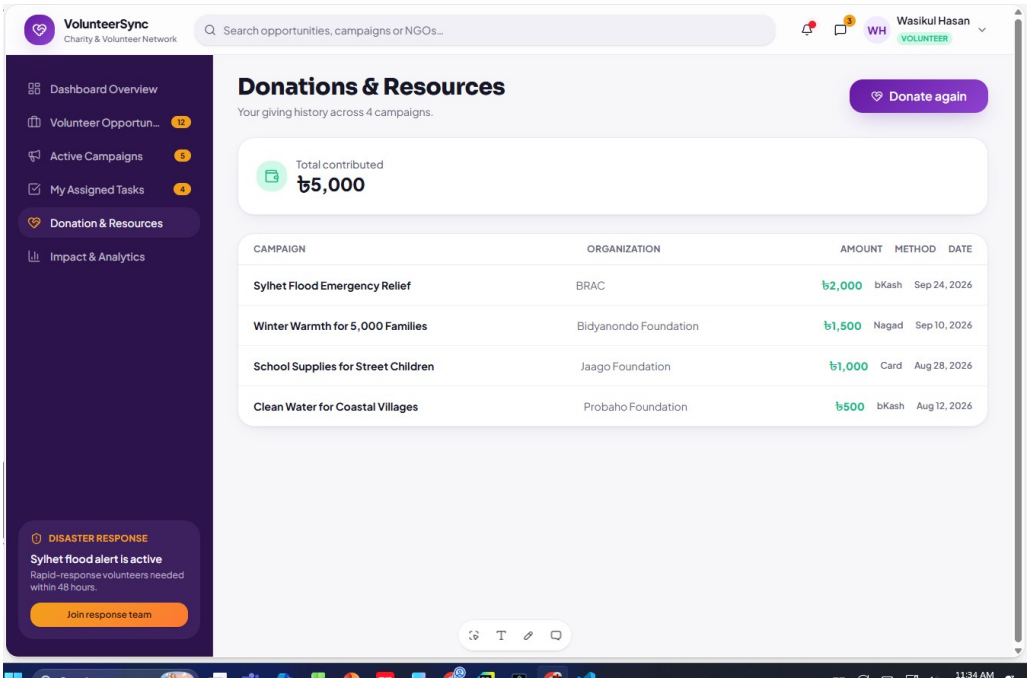


Figure 5.6: Donations and Resources Page

5.4.6 Impact and Analytics Page

The Impact and Analytics page presents volunteer contribution information through summary cards and graphical representations. The page includes:

- Total volunteer hours.
- Number of events attended.
- Number of people reached.
- Number of NGOs supported.
- Monthly volunteer activity.
- Contribution statistics.

This feature helps volunteers understand their social contributions and participation history.

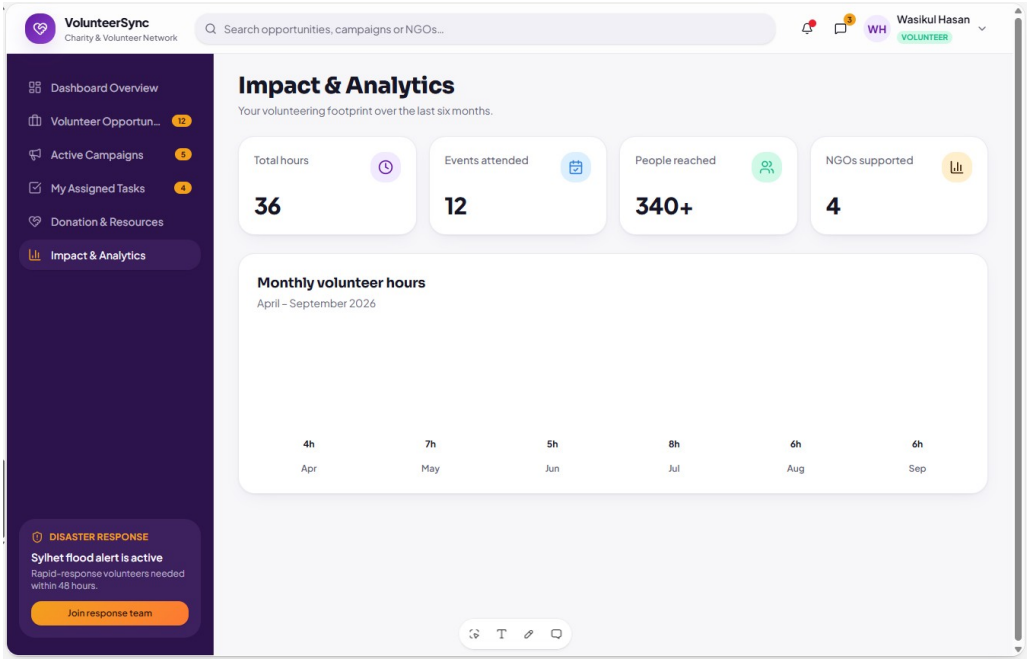


Figure 5.7: Impact and Analytics Page

5.4.7 User Profile and Account Menu

The user profile section provides access to personal information and account-related settings. The menu includes:

- My Account.
- Profile and Skills.
- Volunteer Certificates.
- Notification Settings.
- Logout.

This section helps users manage their account information and preferences.

Chapter 6

API Documentation

6.1 Introduction

API (Application Programming Interface) Documentation describes how different components of the Volunteer & NGO Coordination Platform communicate with each other.

The backend provides API services for user authentication, volunteer management, NGO management, campaign management, task assignment, donation monitoring, and notification services.

The frontend communicates with the backend through HTTP requests. The backend processes the requests, interacts with the MySQL database using SQLAlchemy, and returns appropriate responses. The system uses Python FastAPI to develop backend services.

6.1.1 API Communication

The API communication process follows these steps:

1. The user performs an action through the frontend.
2. The frontend sends an HTTP request to the backend.
3. The backend validates the request.
4. The backend processes the requested operation.
5. The database stores or retrieves the required information.
6. The backend returns a response to the frontend.
7. The frontend displays the result to the user.

6.1.2 HTTP Methods Used

The following HTTP methods are used for different API operations (Table 6.1).

Table 6.1: HTTP Methods Used

HTTP Method	Purpose
GET	Retrieve existing information.
POST	Create new records.
PUT	Update existing records.
PATCH	Partially update records.
DELETE	Remove existing records.

6.2 Module 1: Authentication and User Management

This module manages user registration, login, authentication, and profile information.

6.2.1 User Registration

Purpose

Allows new users to create accounts on the platform.

Method

POST

Proposed Endpoint

`/api/auth/register`

Request Parameters

- Name
- Email
- Password
- Phone
- Role

Expected Response

A confirmation message indicating successful registration or validation errors.

6.2.2 User Login

Purpose

Allows registered users to access the system securely.

Method

POST

Proposed Endpoint
`/api/auth/login`

Request Parameters

- Email
- Password

Expected Response
Authentication status and an access token when authentication succeeds.

6.2.3 User Profile Management

Purpose
Allows authenticated users to view and update their personal information.

Method
GET, PUT

Proposed Endpoints

- `/api/users/profile`
- `/api/users/profile/update`

Expected Response
Updated user profile information.

6.3 Module 2: Volunteer Management

This module manages volunteer profiles, skills, interests, availability, and participation.

6.3.1 Create Volunteer Profile

Purpose
Allows a registered volunteer to create a profile.

Method
POST

Proposed Endpoint
`/api/volunteers`

Request Parameters

- User ID

- Skills
- Interests
- Location
- Availability
- Previous Experience

Expected Response
Volunteer profile creation confirmation.

6.3.2 View Volunteer Information

Purpose
Retrieves volunteer information from the database.

Method
GET

Proposed Endpoint
`/api/volunteers`

Expected Response
A list of registered volunteers.

6.3.3 Update Volunteer Profile

Purpose
Updates existing volunteer information.

Method
PUT

Proposed Endpoint
`/api/volunteers/{volunteer_id}`

Expected Response
Updated volunteer information.

6.3.4 Delete Volunteer Profile

Purpose
Removes a volunteer profile when authorized.

Method
DELETE

Proposed Endpoint
`/api/volunteers/{volunteer_id}`

Expected Response
Deletion confirmation.

6.4 Module 3: NGO Management

This module manages NGO registration, organizational information, verification, and activities.

6.4.1 Register a New NGO

Purpose
Allows an organization to submit its registration information.
Method
POST
Proposed Endpoint
`/api/ngos`
Request Parameters

- NGO Name
- Email
- Contact Number
- Address
- Description
- Registration Information
- Organization Logo

Expected Response
NGO registration confirmation and initial verification status.

6.4.2 View NGO Information

Purpose
Retrieves registered NGO information.
Method
GET
Proposed Endpoint
`/api/ngos`
Expected Response
A list of registered NGOs.

6.4.3 Update NGO Information

Purpose

Allows authorized NGO administrators to update organizational information.

Method

PUT

Proposed Endpoint

`/api/ngos/{ngo_id}`

Expected Response

Updated NGO information.

6.4.4 Delete NGO Information

Purpose

Removes an NGO record when authorized and permitted by related-record constraints.

Method

DELETE

Proposed Endpoint

`/api/ngos/{ngo_id}`

Expected Response

Deletion confirmation or a message explaining why deletion is restricted.

6.4.5 Verify NGO

Purpose

Allows the system administrator to approve or reject NGO verification requests.

Method

PATCH

Proposed Endpoint

`/api/ngos/{ngo_id}/verification`

Request Parameters

- Verification Status
- Admin Remarks

Expected Response

Updated verification status.

6.5 Module 4: Campaign and Event Management

This module manages campaigns and events created by NGOs.

6.5.1 Create a Campaign

Purpose
Allows an NGO to create a new campaign.

Method
POST

Proposed Endpoint
`/api/campaigns`

Request Parameters

- NGO ID
- Campaign Title
- Description
- Location
- Start Date
- End Date
- Required Volunteers
- Campaign Status

Expected Response
Campaign creation confirmation.

6.5.2 View Campaigns

Purpose
Displays available and active campaigns.

Method
GET

Proposed Endpoint
`/api/campaigns`

Expected Response
A list of campaigns.

6.5.3 Update Campaign

Purpose
Updates campaign information.
Method
PUT
Proposed Endpoint
`/api/campaigns/{campaign_id}`
Expected Response
Updated campaign information.

6.5.4 Delete Campaign

Purpose
Removes a campaign when authorized.
Method
DELETE
Proposed Endpoint
`/api/campaigns/{campaign_id}`
Expected Response
Deletion confirmation.

6.6 Module 5: Volunteer Application and Matching

This module handles volunteer applications and suitable opportunity recommendations.

6.6.1 Submit Volunteer Application

Purpose
Allows volunteers to apply for a campaign or opportunity.
Method
POST
Proposed Endpoint
`/api/applications`
Request Parameters

- Volunteer ID
- Campaign ID

- Application Date
- Status

Expected Response

Application submission confirmation.

6.6.2 View Applications

Purpose

Allows NGOs to view submitted applications.

Method

GET

Proposed Endpoint

`/api/applications`

Expected Response

A list of volunteer applications.

6.6.3 Update Application Status

Purpose

Allows NGO administrators to accept or reject volunteer applications.

Method

PATCH

Proposed Endpoint

`/api/applications/{application_id}/status`

Expected Response

Updated application status.

6.6.4 Volunteer Opportunity Matching

Purpose

Provides suitable opportunities based on volunteer information.

Method

GET

Proposed Endpoint

`/api/matching/opportunities`

Matching Criteria

- Skills

- Interests
- Location
- Availability
- Required Skills

Expected Response
A list of suitable volunteering opportunities.

6.7 Module 6: Task and Participation Management

This module manages task assignment, task progress, and volunteer participation.

6.7.1 Create a Task

Purpose
Allows NGO administrators to create and assign a task.

Method
POST

Proposed Endpoint
`/api/tasks`

Request Parameters

- Campaign ID
- Volunteer ID
- Task Name
- Description
- Deadline
- Status

Expected Response
Task creation confirmation.

6.7.2 View Assigned Tasks

Purpose
Retrieves assigned tasks for authorized users.

Method
GET

Proposed Endpoint
`/api/tasks`

6.7.3 Update Task Status

Purpose
Updates task progress. Possible status values: Pending, Accepted, In Progress, Completed.

Method
PATCH

Proposed Endpoint
`/api/tasks/{task_id}/status`

6.7.4 Record Volunteer Attendance

Purpose
Stores volunteer attendance and participation information.

Method
POST

Proposed Endpoint
`/api/participations`

6.8 Module 7: Donation and Resource Management

This module maintains donation information and monitors resource contributions.

6.8.1 Create Donation Record

Purpose
Creates a donation record.

Method
POST

Proposed Endpoint
`/api/donations`

Request Parameters

- Donor ID
- NGO ID

- Campaign ID
- Donation Amount
- Donation Type
- Donation Date
- Payment Status

Expected Response

Donation record creation confirmation.

6.8.2 View Donation Records

Purpose

Retrieves donation information for authorized users.

Method

GET

Proposed Endpoint

`/api/donations`

6.8.3 Update Donation Information

Purpose

Updates authorized donation records.

Method

PUT

Proposed Endpoint

`/api/donations/{donation_id}`

6.8.4 Resource Management

Purpose

Manages non-financial resources such as food, clothes, medicine, and educational materials.

Method

POST, GET, PUT, DELETE

Proposed Endpoint

`/api/resources`

6.9 Module 8: Beneficiary Management

This module maintains information about individuals and communities requiring assistance.

6.9.1 Register Beneficiary

Purpose
Registers a beneficiary who requires assistance.

Method
POST

Proposed Endpoint
`/api/beneficiaries`

Request Parameters

- Name
- Contact
- Location
- Required Assistance
- Campaign ID
- Support Status

Expected Response
Beneficiary registration confirmation.

6.9.2 View Beneficiaries

Purpose
Retrieves beneficiary records for authorized NGO administrators.

Method
GET

Proposed Endpoint
`/api/beneficiaries`

6.9.3 Update Beneficiary Information

Purpose
Updates beneficiary details and support status.

Method
PUT

Proposed Endpoint
`/api/beneficiaries/{beneficiary_id}`

6.10 Module 9: Notification and Communication

This module provides important updates and notifications to system users.

6.10.1 Create Notification

Purpose
Creates notifications for campaign updates, task assignments, application status, and emergency announcements.

Method
POST

Proposed Endpoint
`/api/notifications`

6.10.2 View Notifications

Purpose
Retrieves notifications for authenticated users.

Method
GET

Proposed Endpoint
`/api/notifications`

6.10.3 Real-Time Communication

Purpose
Supports real-time notification delivery and communication using WebSocket technology.

Method
WebSocket

Proposed Connection
`/ws/notifications`

6.11 Module 10: Administration and Auditing

This module provides administrative control and maintains records of important system activities.

6.11.1 Manage Users

Purpose
Allows authorized administrators to monitor and manage user accounts.

Method
GET, PATCH, DELETE

Proposed Endpoint
`/api/admin/users`

6.11.2 View Audit Logs

Purpose
Retrieves important administrative and resource-management activities.

Method
GET

Proposed Endpoint
`/api/admin/audit-logs`

6.11.3 Generate Reports

Purpose
Provides reports related to volunteers, campaigns, donations, and resource utilization.

Method
GET

Proposed Endpoint
`/api/admin/reports`

6.12 API Security

The API layer should follow appropriate security practices:

- JWT-based authentication.
- Role-based access control.

- Request validation.
- Secure password hashing.
- Restricted access to sensitive information.
- Proper error handling.
- Secure file upload validation.
- HTTPS/TLS for communication.
- Audit logging for important administrative activities.

Chapter 7

Testing and Sustainability Plan

7.1 Introduction

Testing is an important part of software development. It helps identify errors, verify system functionality, and ensure that the application meets the specified requirements.

The Volunteer & NGO Coordination Platform requires testing of its authentication, volunteer management, NGO management, campaign management, donation management, and communication features.

The sustainability plan focuses on maintaining the system, protecting information, improving performance, and supporting future development.

7.2 Testing Strategy

The following testing approaches are considered for the platform.

7.2.1 Unit Testing

Unit testing verifies individual system components independently. The following components should be tested:

- User registration and login.
- Volunteer profile creation.
- NGO registration.
- Campaign creation.
- Volunteer application submission.
- Task assignment.
- Donation record creation.

7.2.2 Integration Testing

Integration testing verifies whether different modules work together correctly. Examples include:

- User authentication with profile management.
- NGO management with campaign management.
- Volunteer management with opportunity matching.
- Campaign management with donation records.
- Task management with volunteer participation.
- Notification services with user activities.

7.2.3 Functional Testing

Functional testing verifies whether the system performs its required operations. The main functions include:

- Registering and logging in users.
- Creating, viewing, updating, and deleting records.
- Searching for volunteer opportunities.
- Applying for campaigns.
- Assigning tasks.
- Managing donations and resources.
- Viewing notifications.

7.2.4 Security Testing

Security testing evaluates the protection of user information and system resources. The following areas should be checked:

- Authentication and authorization.
- Role-based access control.
- Password protection.
- Unauthorized access prevention.
- Input validation.
- File upload validation.
- Protection of beneficiary information.

7.2.5 Usability Testing

Usability testing evaluates whether users can easily interact with the platform. The interface should be checked for:

- Simple navigation.
- Clear information presentation.
- Responsive design.
- Readable text and buttons.
- Proper form validation.
- User-friendly dashboards.

7.3 Test Cases

The major test cases of the platform are presented in Table 7.1.

Table 7.1: Functional Test Cases

Test ID	Test Scenario	Expected Result
TC-01	User registration with valid information	Account should be created.
TC-02	Login with valid credentials	User should access the dashboard.
TC-03	Login with invalid credentials	Appropriate error message should appear.
TC-04	Create a volunteer profile	Profile information should be saved.
TC-05	Register an NGO	NGO information should be stored.
TC-06	Create a campaign	Campaign should appear in the campaign list.
TC-07	Apply for an opportunity	Application should be recorded.
TC-08	Update application status	New status should be displayed.
TC-09	Assign a task to a volunteer	Task should appear in the volunteer dashboard.
TC-10	Create a donation record	Donation information should be stored.
TC-11	Update task status	Updated task status should be displayed.
TC-12	Access restricted administrative information	Unauthorized access should be prevented.

7.4 Sustainability Plan

The sustainability plan ensures that the platform remains useful, secure, and maintainable over time.

7.4.1 Technical Sustainability

- Regularly update the frontend and backend technologies.
- Maintain the MySQL database.
- Fix technical errors and software bugs.
- Improve system performance.
- Maintain compatibility with modern browsers and devices.

7.4.2 Data Sustainability

- Maintain regular database backups.
- Protect sensitive user information.
- Maintain accurate donation records.
- Keep volunteer and campaign information updated.
- Implement appropriate data recovery procedures.

7.4.3 Operational Sustainability

- Provide simple user interfaces.
- Support NGO administrators with system guidance.
- Maintain clear user roles and responsibilities.
- Monitor system performance.
- Provide technical support when necessary.

7.4.4 Financial Sustainability

- Use open-source technologies where possible.
- Minimize unnecessary hosting and maintenance expenses.
- Explore partnerships with NGOs and social organizations.
- Consider sponsorship and institutional support for future deployment.

7.4.5 Future Development

The platform can be improved through the following enhancements:

- Mobile application development.
- Advanced AI-based volunteer matching.
- Online donation payment gateway integration.
- Digital volunteer certificates.
- Advanced reporting and analytics.
- Location-based opportunity recommendations.
- Improved emergency notification services.

7.5 Conclusion

The testing and sustainability plan provides a structured approach to evaluating the reliability, security, and usability of the Volunteer & NGO Coordination Platform.

By performing appropriate testing and maintaining regular system updates, backups, and security practices, the platform can provide reliable services to volunteers, NGOs, donors, and beneficiaries.

Chapter 8

CEP Justification

8.1 Introduction

The Volunteer & NGO Coordination Platform contributes to community development by connecting volunteers, NGOs, donors, and beneficiaries through a centralized digital system.

The project focuses on improving volunteer participation, supporting humanitarian activities, increasing donation transparency, and providing organized assistance to communities.

8.2 Community Engagement

The platform encourages individuals to participate in different social and humanitarian activities. Volunteers can discover opportunities and participate in activities according to their skills, interests, and availability.

NGOs can organize community programs, manage volunteers, and coordinate different campaigns through the platform.

8.3 Social Contribution

The project supports community development in several ways:

- Encourages volunteer participation in social activities.
- Helps NGOs organize relief and assistance campaigns.
- Connects donors with social campaigns.
- Helps identify beneficiaries who require assistance.
- Improves communication between volunteers and organizations.
- Supports educational, environmental, and humanitarian activities.

8.4 Environmental Contribution

The platform supports environmental sustainability by encouraging digital documentation and reducing unnecessary paper-based processes.

It can also support environmental campaigns such as tree plantation, waste management, and community cleaning activities.

8.5 Professional and Technical Development

The project provides practical experience in software engineering and collaborative development. The team members gain experience in:

- Software requirement analysis.
- System architecture and database design.
- Frontend and backend development.
- API development and integration.
- Database management.
- Software testing.
- Technical documentation.
- Teamwork and project management.

8.6 Conclusion

The Volunteer & NGO Coordination Platform demonstrates how technology can support community engagement and social development.

The project provides a digital environment for coordinating volunteers, NGOs, donors, and beneficiaries. It encourages social responsibility, improves organizational coordination, and supports transparent management of donations and resources.

Therefore, the project has both social value and educational importance.

Chapter 9

Conclusion

The Volunteer & NGO Coordination Platform is a proposed centralized digital solution designed to improve communication and coordination among volunteers, NGOs, donors, and beneficiaries.

The project focuses on solving common challenges in volunteer management, NGO activity coordination, donation monitoring, resource distribution, and beneficiary support.

The system provides several important functionalities, including user authentication, volunteer profile management, NGO management, campaign and event management, volunteer opportunity matching, task assignment, donation monitoring, notification services, and administrative control.

The platform uses modern web development technologies, including React.js, Tailwind CSS, Python FastAPI, MySQL, SQLAlchemy, JWT, and WebSocket. These technologies provide the foundation for developing a scalable, secure, and user-friendly application.

Through this project, the team gains practical experience in software engineering, database design, application development, testing, teamwork, and technical documentation.

In the future, the platform can be enhanced with advanced volunteer matching, mobile application support, online payment integration, digital certificates, and improved analytical features.

Overall, the Volunteer & NGO Coordination Platform aims to strengthen volunteer engagement, improve NGO efficiency, increase donation transparency, and contribute to community development in Bangladesh.